Understanding the Problem:

The "Escape from Castle" game is a text-based game where the player is trapped in a castle and must find a way to escape. The player will be given a map of the castle and will be able to move around the castle. The player will be able to pick up programming skill and use it to help them escape. The player will also be able to battle the TAs and the Instructor in the castle. The player will be able to win the game by proving their skill to the Insructor the castle or lose the game by failing their assignments.

Program Design:

Requirements:

```
1. The player must be able to move around the castle.
2. The player must be able to pick up programming skill.
3. The player must be able to battle the TAs and the Instructor.
4. The player must be able to win the game by proving their skill to the Instructor.
5. The player must be able to lose the game by failing their assignments.
```

Game Flow:

```
1. Start with a game menu. (Start, Leaderboard, Quit)
2. Start the game.
3. The player will be given a map of the castle.
4. The player will be able to move around the castle.
5. The player will be able to pick up programming skill.
6. When the player picks up programming skill, the player will demonstrate their skill to get the point.
7. The TAs randomly move around the castle.
8. When the player and the TAs are in the same room, the player will battle the TAs.
9. When the player and the Instructor are in the same room, the player will battle the Instructor.
10. The player will win the game by proving their skill to the Instructor.
11. The player will lose the game by failing their assignments.
12. The player will go back to the game menu.
```

Classes:

```
 Maze Person:
Abstract class that contains the basic information for the player and the TAs and the Instructor.
- int x
- int y
- int skill
- virtual void move(direction)
- virtual void battle()

Intrepid Student:
The player of the game. The player will be able to move around the castle, pick up programming skill, and battle the TAs and the Ins
- void move(direction)

TA:
The TAs will be able to move around the castle and battle the player.
- void move(direction)
- void battle()

Instructor:
The Instructor will stand still and battle the player.
- void battle()
```

```
Game Manager:
The Game Manager will hold all the rooms in the castle and will be able to start the game.
- Maze maze;
- Student* player;
- void startGame()
- void startMenu()
- void displayLeaderboard()
- void displayGameMenu()
- void displayBattleMenu()
- void displayWin()
- void displayLose()
- void displayQuit()

Maze:
vector<vector<Room*>> locations;
- void movePerson(MazePerson*, direction)
- void addPerson(MazePerson*, int, int)
- void addRoom(Room*, int, int)
- void displayMap()

Room:
The Room will hold the information for each room in the castle.
- vector<MazePerson*> people;
- bool hasSkill
- void addPerson(MazePerson*)
- void removePerson(MazePerson*)
- void displayRoom()

Wall Room:
The Wall Room will be the walls of the castle.
- void displayRoom()

Open Room:
The Open Room will be the rooms that the player can move around in.
- void displayRoom()
```

Pseudocode:

Pseudocode for the Game Manager:

```
startMenu()
    displayMenu()
    getChoice()
    if(choice == 1)
        startGame()
    else if(choice == 2)
        displayLeaderboard()
    else if(choice == 3)
        displayQuit()

startGame()
    createMaze()
    createPlayer()
    while(!complete)
        displayGameMenu()
        getChoice()
        displayMap()

        getMove()
        movePerson(player, move)
        if(maze->playerBattle(player))
            displayBattleMenu()

        maze->moveTAs()
```

Pseudocode for the Maze:

```
movePerson(MazePerson*, direction)
    if(direction == up)
        if(y - 1 >= 0)
            if(locations[y - 1][x]->isWall())
                return false
            else
                locations[y][x]->removePerson(person)
                locations[y - 1][x]->addPerson(person)
                y--
                return true
        else
            return false
    else if(direction == down)
        if(y + 1 < locations.size())
            if(locations[y + 1][x]->isWall())
                return false
            else
                locations[y][x]->removePerson(person)
                locations[y + 1][x]->addPerson(person)
                y++
                return true
        else
            return false
    else if(direction == left)
        if(x - 1 >= 0)
            if(locations[y][x - 1]->isWall())
                return false
            else
                locations[y][x]->removePerson(person)
                locations[y][x - 1]->addPerson(person)
                x--
                return true
        else
            return false
    else if(direction == right)
        if(x + 1 < locations[y].size())
            if(locations[y][x + 1]->isWall())
                return false
            else
                locations[y][x]->removePerson(person)
                locations[y][x + 1]->addPerson(person)
                x++
                return true
        else
            return false
```

Pseudocode for the Room:

```
addPerson(MazePerson*)
    people.push_back(person)

removePerson(MazePerson*)
    for(int i = 0; i < people.size(); i++)
        if(people[i] == person)
            people.erase(people.begin() + i)
            break
```

Pseudocode for the Maze Person:

```
move(direction)
    maze->movePerson(this, direction)

battle()
    if(this->skill > person->skill)
        person->skill = 0
    else
        this->skill = 0
```

Pseudocode for the Intrepid Student:

```
move(direction)
    if(maze->movePerson(this, direction))
        if(maze->locations[y][x]->hasSkill())
            battle()
```

Pseudocode for the TA:

```
move(direction)
    maze->movePerson(this, direction)
```

Pseudocode for the Instructor:

```
battle()
    if(this->skill > person->skill)
        person->skill = 0
        complete = true
    else
        this->skill = 0
```

Proposed Test Cases:

```
1. Test that the player can move around the castle.
- Test that the player can move up. (y - 1)
- Test that the player can move down. (y + 1)
- Test that the player can move left. (x - 1)
- Test that the player can move right. (x + 1)

2. Test that the player can pick up programming skill.
- Test that the player can pick up skill in an open room.
- Test that the player cannot pick up skill in a wall room.

3. Test that the player can battle the TAs.
- Test that the player can win the battle.
- Test that the player can lose the battle.

4. Test that the player can battle the Instructor.
- Test that the player can win the battle.
- Test that the player can lose the battle.

5. Test that the player can win the game.
- Test that the player can win the game by defeating the Instructor.

6. Test that the player can lose the game.
- Test that the player can lose the game by losing to the TAs.
- Test that the player can lose the game by losing to the Instructor.
```